

Week 22 | Audit transfer before calling it a foundation model

A capstone that extends Week 7.4

Week 7.4 already introduces pretraining on diverse wake trajectories. Repeating a smaller source-to-target demonstration would not advance the course. This capstone instead asks students to audit why an apparent transfer benefit occurs: initialization, access to source information, representation quality, extra fitting effort or an inconsistent target-label budget.

The release uses source Re90 and three retained targets, Re100, Re105 and Re110. Those trajectories have appeared in earlier teaching and model selection. They are therefore mechanism-audit cases, not new blind research tests. There is no claim of a multi-equation foundation model or generalization to an unseen physical family.

Fix forecast horizon and count every label

Every arm begins from observed target frames 156:160, predicts 160:281 autonomously and is scored on 210:281. Varying the amount of adaptation data does not change that start or horizon. Label budgets consist of predetermined nonoverlapping ten-frame blocks before frame 150, plus the four initialization frames.

Block budget	Training frames	Validation frames	Initialization frames	Total target labels
4	30	10	4	44
8	60	20	4	84
15	120	30	4	154

Training blocks are spread across the development interval; validation blocks lie later and do not overlap them. Windows are formed inside each block, so the learner cannot create a transition across an unobserved gap. Source pretraining labels are a separate resource and remain in the accounting rather than disappearing into the word pretrained.

What each arm isolates

The scratch and pretrained causal decoders share the source-only POD basis and coefficient scaling. Their comparison isolates dynamics initialization conditional on that representation. Scratch is not source-free: it still benefits from the source-fitted basis. A matched-step decoder receives the source-plus-adaptation update ceiling entirely on target training data. A target-POD MLP provides a representation control using a basis fitted only on target training labels. DMD uses adjacent pairs inside the same allowed training blocks.

The target-POD MLP changes both representation and architecture, so its result cannot attribute all improvement to the basis alone. It is a diagnostic control: a much lower target-POD floor reveals a

representation bottleneck and motivates a follow-on same-architecture representation ablation. This qualification belongs beside the result, not only in a remote limitations section.

Compute matching has more than one meaning

Three hundred source steps plus three hundred adaptation steps equal six hundred optimizer updates. They do not equal the floating-point operations or wall time of six hundred target steps: source and target batches have different sizes, and the MLP and Transformer have different parameter counts. The record therefore reports executed updates, selected checkpoint, parameter count and measured fitting time.

$$C_{\text{deployment}} = C_{\text{adapt}} + \frac{C_{\text{source}}}{N_{\text{targets}}}$$

Amortizing source cost requires a declared number of target deployments. For one target, charge the complete source cost. For many targets, show both one-time and marginal costs. Never use amortization to hide that source data were unavailable to a supposedly information-matched source-free baseline.

The fixed update budgets make the compute-control question reproducible. A checkpoint at the ceiling remains visibly flagged. These experiments do not prove that every arm is globally optimized, and a budget extension should be selected by development criteria rather than by retained evaluation rankings.

Separate representation from adaptation

A source-trained basis may represent the target wake poorly. Projecting target truth into that basis gives a lower bound on field error for every model that uses its decoder. If this floor is 25.59% and total field error is 25.60%, the neural dynamics are not responsible for almost all of the remaining field discrepancy.

$$R = E_{\text{representation}}^2 / E_{\text{field}}^2$$

The laboratory flags representation-dominated rows when this ratio exceeds 0.95. That threshold is a teaching diagnostic, not a universal law. A plot should show field error and in-subspace error side by side, with the source-POD floor on the field panel. The target-POD control has its own floor; drawing one floor for all representations would be misleading.

The error decomposition uses squared norms. Subtracting 25.59% from 25.60% does not recover a dynamics norm. The actual in-subspace error must be measured against the projected truth or obtained from the square root of a nonnegative squared-error difference when the orthogonality assumptions hold.

In the retained 44-label regime, 34 of 36 neural fits selected step zero. All fresh-initialization arms, including target-POD-MLP, are flagged as untrained if they select zero; a pretrained step-zero arm instead retains source weights without target adaptation. All three source fits selected 300/300. Their 50-step validation criterion therefore did not change the final source checkpoint in these runs.

Source and target window coverage, six-step target validation and update budgets differ; the experiment does not isolate which difference causes a gain.

A bounded transfer statement

Define relative gain as one minus pretrained in-subspace error divided by matched-control in-subspace error. Positive gain means lower error in that particular comparison; negative gain is negative transfer and remains in the table. A denominator near zero makes the ratio unstable, so absolute errors should accompany it.

$$G = 1 - E_{pretrained}/E_{control}$$

A valid conclusion names target case, target-label budget, representation and compute control. For example: on one retained target and the 84-label protocol, source initialization reduced in-subspace error relative to a declared control. That statement does not establish universal sample efficiency, full-field superiority or broad foundation-model behavior.

Aggregate means over seeds should retain their ranges. The three targets are related trajectories of the same coarse solver and geometry; they are not independent samples from all fluid mechanics. Do not turn the number of rows in a results table into an inflated count of independent scientific cases.

Reproduction as an executable claim

Stored checkpoints include the model, exact POD basis, mean and scale. Stored prediction factors reconstruct every retained transfer trajectory within a stated numerical tolerance. Their low-rank storage factors are merely a serialization device, distinct from the training POD representation. The notebook reconstructs the W22 trajectories and recomputes W22 scores directly rather than displaying a Week 21 metric under a transfer heading.

The manifest records all local scientific implementation hashes, input hashes, the protocol, platform and package versions. Text hashes normalize line endings; binary arrays retain byte hashes. A fresh-run verifier must compare numerical results with tolerances, while a checksum verifier only establishes that an artifact has not changed. These are different tests.

Capstone submission and research reading

Submit a label-accounting table, compute table, three-target comparison, field/subspace plots and a claim ledger listing supported, unsupported and unresolved statements. The four coding tasks count label unions, account for updates, detect floor-dominated errors and audit paired transfer comparisons, including separate checkpoint and boundary flags. The written component should identify the next control that would most reduce uncertainty, rather than merely recommending a larger model.

Herde et al., Poseidon: Efficient Foundation Models for PDEs (2024), <https://arxiv.org/abs/2405.19101>, is a research reading for comparing scope, pretraining distributions and downstream evidence. Wu et al., Transolver (2024), <https://arxiv.org/abs/2402.02366>, supports an architectural comparison with Week 15. Neither

published model is reproduced by this course experiment. Students should explain what additional datasets, controls and resource accounting would be required before making a comparable claim.

Printable laboratory worksheet

Task 1: Audit information budgets

Return the cardinality of the union of all target frames used before scoring.

Interface contract: record has `training_frames`, `validation_frames` and `initialization_frames`, each an iterable of integer frame IDs. Return an integer cardinality of their union, counting every overlap once.

Task 2: Match compute accounting

Count source pretraining and target adaptation steps for each learned arm, without charging source cost to a source-free arm.

Interface contract: `adaptation` is a dictionary with `executed_steps` (nonnegative integer). `pretraining` is `None` or a dictionary with `executed_steps`. Return their sum when `pretraining` is supplied, otherwise `adaptation_executed_steps`.

Task 3: Detect a representation-limited result

Flag cases where the squared representation error explains more than 95% of squared field error. Do not subtract unsquared norms.

Interface contract: `metrics` is a dictionary with `representation_floor` (floor) and `field_relative_l2` (field), both nonnegative scalar relative L2 errors. Return a bool for $\text{floor}^2 / \max(\text{field}^2, 1e-20) > 0.95$. At zero/zero return `False`; use a strict threshold.

Task 4: Build a defensible transfer comparison

For records with key (target, labels, seed), pair pretrained and matched-steps arms. Reject missing or duplicate arms and nonpositive control error. Return signed relative gain, selected-step flags and budget-boundary flags. Treat step-zero scratch/matched results as untrained controls. Apply the audit to a normalized copy of the retained table; explain why this does not isolate representation or validation-horizon effects.

Interface contract: Each input dictionary has `target`, `labels` (integer total target labels), `seed`, `arm`, `error` (in-subspace relative L2), `selected_step`, `ceiling`. Pair pretrained with matched-steps per (target,labels,seed), rejecting duplicates/missing arms or nonpositive control error with `ValueError`. Return a list sorted by group key of dictionaries: key (tuple), `gain` ($1 - \text{pretrained_error} / \text{control_error}$), `control_untrained` (control selected step==0), `pretrained_selected_step`, `control_selected_step`, `pretrained_boundary` and `control_boundary` (each arm selected step==its own ceiling). Keep negative gains.

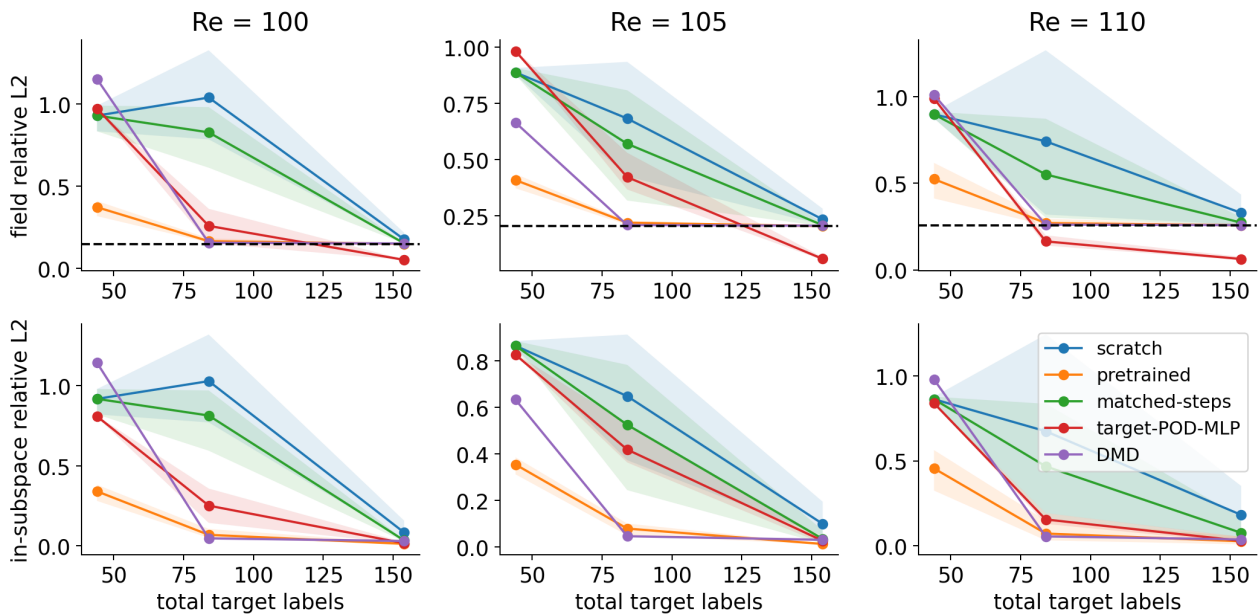
For the integrated fourth task, compare at least two controlled settings and explain a result that the implementation alone cannot justify. Include the requested plot or table and retain unsuccessful outcomes.

Before submitting

Restart the kernel and run the worked examples and completed tasks. Confirm that all four coding checks pass and that no retained data or results file changed. Include the requested interpretation, identify every quantity fitted from training data, and separate test observations from decisions made using validation. Record unresolved budget or representation limits rather than removing inconvenient runs.

Executed evidence and interpretation

Fixed-start transfer: mean and seed range, not confidence intervals



44 labels: 25 fresh-initialization fits selected step 0; 9 pretrained fits used no target adaptation.

This figure is generated from the retained protocol. Use the notebook to inspect all seeds and recompute the plotted quantities. The earlier lecture sections specify what the diagnostic can and cannot establish.